Natural Language Processing

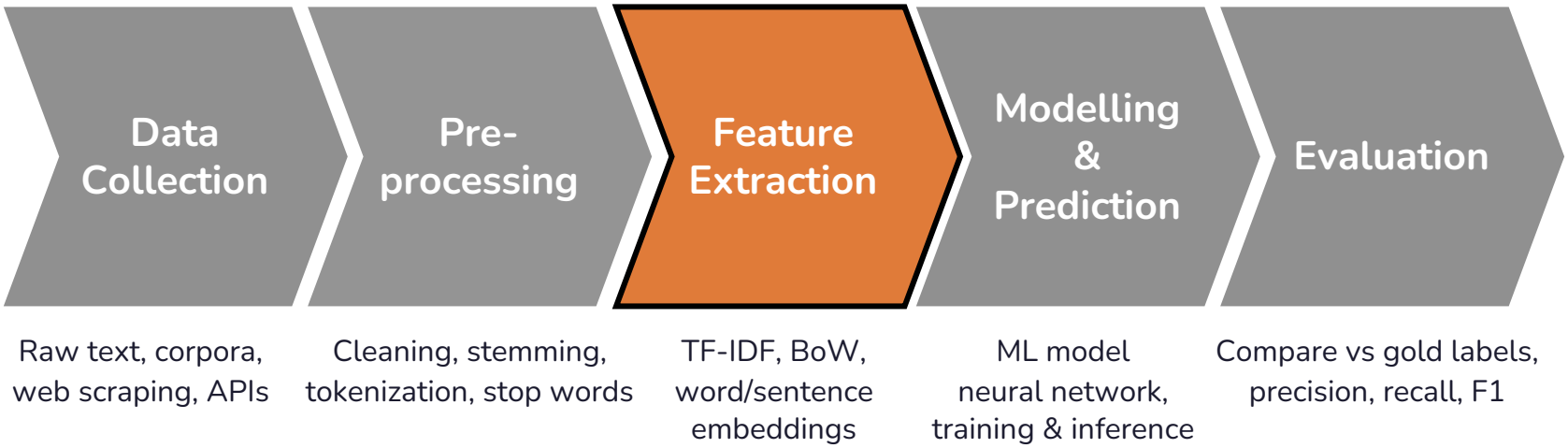
Session 04 – Text Vectorization

Prof. Dr. Richard Sieg
SoSe 2026

Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 **Exams — Master DS Students**
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Recall: The typical NLP pipeline



Agenda

01. Recap: Vectors and sklearn

02. Bag of Words

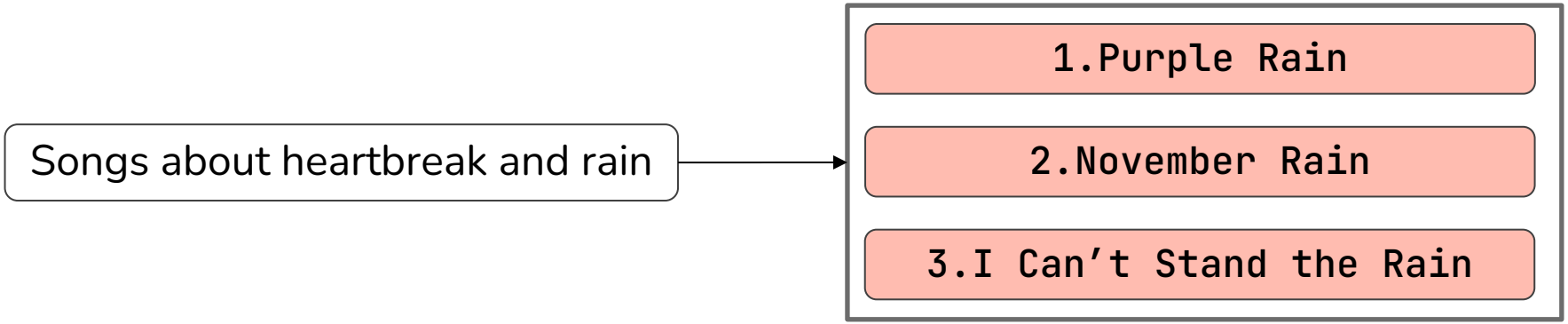
03. TF-IDF

04. BM25

Two Things We Want to Do

Classification: Given a song's lyrics, predict its genre.

Retrieval: Given a query, find the most relevant documents.



Why Numbers?

Machines cannot compute with words directly.

```
"I love NLP" + "NLP is great" = ???
```

But they are very good at computing with vectors:

```
[0.3, 0.8, 0.1] + [0.1, 0.7, 0.9] = [0.4, 1.5, 1.0]
```

How do we represent a document as a vector of numbers?

01.

Vector Operations & scikit-learn

Vector Operations

- Vectors are essentially a list of n (=dimensions) entries of (real) numbers
- Appear everywhere in machine learning (e.g. layers in neural networks)
- Vectors can be added and multiplied by scalars:

$$2 \cdot \begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} + 3 \cdot \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 9 \end{pmatrix}$$

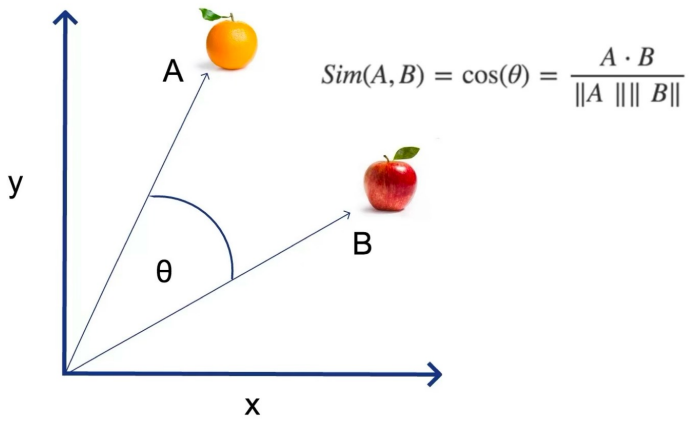
- Inner product / dot product between two vectors:

$$\begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix} = 2 \cdot 0 + (-1) \cdot 2 + 3 \cdot 1 = 1$$

- Norm / length of a vector: $\left\| \begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} \right\| = \sqrt{2^2 + (-1)^2 + 3^2} = \sqrt{14}$

Measuring Similarity

- The dot product measures how much two vectors point in the same direction…
  - …but it is biased by vector length: longer vectors always score higher, regardless of direction
  - e.g. a 10,000-word document scores higher than a 50-word one, even if less relevant
- Solution: normalize by vector length → measures angle only, not magnitude
- Angle: $\cos(v, w) = \frac{v \cdot w}{\|v\|\|w\|}$ (value between -1 and 1)
- Small angle means: value close to 1
- In NLP we call this ***Cosine Similarity***
- Sometimes also used: $1 - \cos(v, w)$



Example

Three words and their vectors are their co-occurrences with other words:
pie, *data*, *computer*

	pie	**data**	**computer**
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

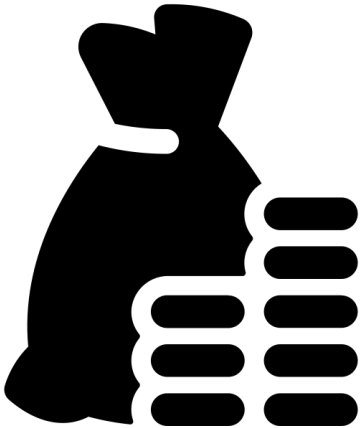
cos (cherry, information) = 0.018 → very different

cos (digital, information) = 0.996 → very similar

scikit-learn tutorial in marimo notebook

02.

Bag-of-Words



Created by Gregor Cresnar
from Noun Project

The Problem

We want to turn this:

"I got my mind on my money and my money on my mind"

into a ***fixed-length vector of numbers***:

$$[1,1,1,2,2,4,2]$$

Simple idea: assign each word an index and represent a document by which words appear and how often.

Step 1: Build a Vocabulary

Collect all unique words across all documents in the corpus and assign each word a unique index.

Take on me

Take me on

Take my breath away

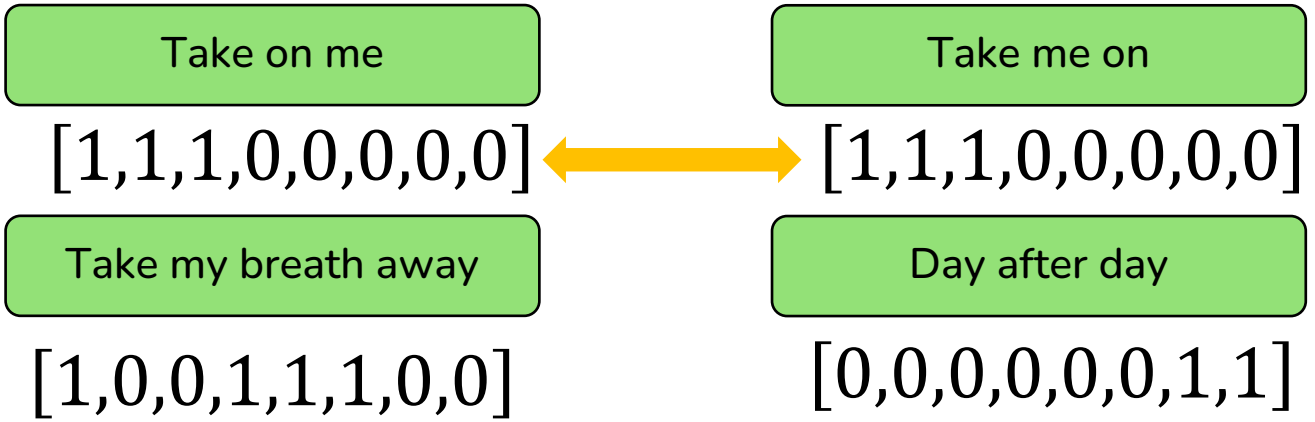
Day after day

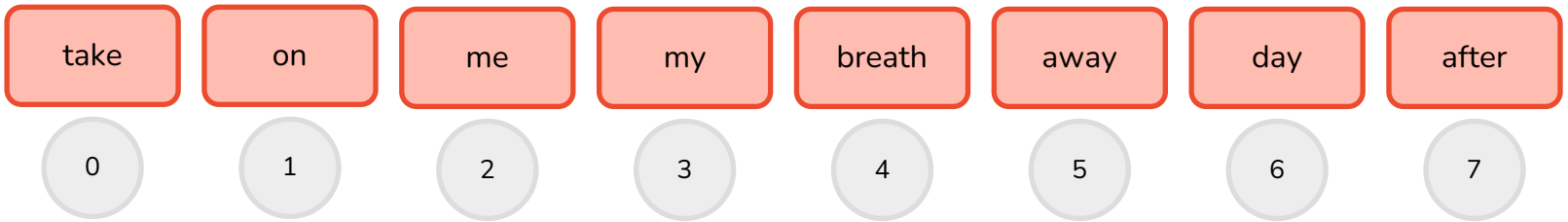
Vocabulary (lowercase)

take	on	me	my	breath	away	day	after
0	1	2	3	4	5	6	7

Step 2: Vectorize Each Document

For each line, create a vector of length |V| = 8. The vector is 1 when the word appears and 0 if not.






take on me
take me on
Corporate needs you to find the differences between this picture and this picture.
BoW
They're the same picture.
imgflip.com

Binary vs. Frequency

"Day after day" contains the word "day" twice. Does our vector capture that?

Binary (does the word appear?):

Day after day → $[0,0,0,0,0,0,1,1]$

Frequency (how often does the word appear?):

Day after day → $[0,0,0,0,0,0,2,1]$

The Document-Term Matrix

Stack all document vectors into a matrix:

- Rows = documents
- Columns = vocabulary terms
- Cells = word counts (or binary indicators)

This is called the ***document-term matrix***

$$\begin{array}{c} \\ doc_1 \\ doc_2 \\ doc_3 \end{array} \begin{array}{c} \begin{array}{cccccc} term_1 & term_2 & term_3 & & & term_n \end{array} \\ \begin{bmatrix} 1 & 0 & 3 & \cdots & 0 \\ 0 & 2 & 1 & \cdots & 1 \\ 4 & 0 & 0 & \cdots & 2 \end{bmatrix} \end{array}$$

Sparse Matrix

- The typical vocabulary has 10k to 50k words and a document uses only up to 500 of them.

→ Most entries are zeros.

→ We only store the non-zero entries.

Building a Good Vocabulary

scikit-learn parameter	What it does	Example
Lowercasing	"Love" and "love" become the same token	On by default
Stop words	Remove very frequent words (the, is, a, ...)	stop_words='english'
min_df	Ignore words that appear in fewer than N documents	min_df=2 removes typos and ultra-rare words
max_df	Ignore words that appear in more than X% of documents	max_df=0.95 removes corpus-wide noise
max_features	Keep only the top N most frequent words	max_features=5000 caps vocabulary size

A larger vocabulary captures more detail but creates larger, sparser vectors and may include noise. A smaller vocabulary is more robust but may miss informative words.

N-grams

So far, each feature is a single word (unigram).

But sometimes word combinations carry meaning that single words do not:

- Bigrams: “not good” is one feature
- Trigrams: “not very good” is one feature
- N-gram: a contiguous sequence of N tokens

In scikit-learn **`ngram_range=(1,2)`** means ”use unigrams and bigrams”

N	Features
Unigrams (1)	I, do, not, like, green, eggs (6)
Bigrams (2)	I do, do not, not like, like green, green eggs (5)
Trigrams (3)	I do not, do not like, not like green, like green eggs (4)

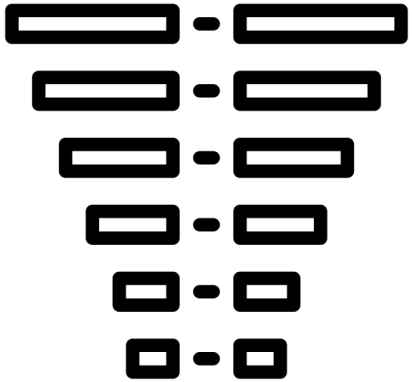
Vocabulary size increases dramatically.

↓

Use **`max_features`**

03.

tf-idf



Created by iconixar
from Noun Project

The Problem with Raw Counts

- Consider a corpus of 8k song lyrics.
- The word "the" appears in almost every song. It will dominate every BoW vector unless removed.
- The word "heartbreak" appears in only a few songs. It is much more informative about what a song is about.
- Raw counts treat both words equally. We need a way to downweight common words and upweight rare, distinctive ones.

Karen Spärck Jones

In 1972, Karen Spärck Jones published a paper that solved exactly this problem:

"A Statistical Interpretation of Term Specificity and its Application in Retrieval"

- Her idea: weight terms by how rare they are across the collection. Common words get low weight. Rare words get high weight.
- This concept is called ***Inverse Document Frequency (IDF).***
- Combined with term frequency, it became TF-IDF, the foundation of modern search engines.

Term Frequency (TF)

How important is a token within this document?

Term Frequency $\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$ – the relative frequency of the token t in document d

Take on me

$[1/3, 1/3, 1/3, 0,0,0,0,0]$

Take me on

$[1/3, 1/3, 1/3, 0,0,0,0,0]$

Take my breath away

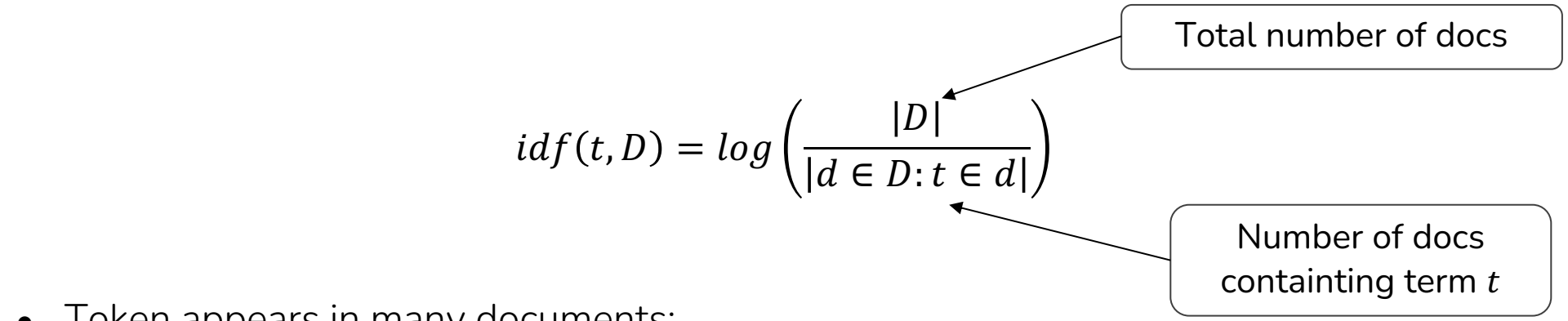
$[1/4, 0,0, 1/4, 1/4, 1/4,0,0]$

Day after day

$[0,0,0,0,0,0, 2/3, 1/3]$

Inverse Document Frequency (IDF)

IDF measures how rare a word is across the entire corpus.



- Token appears in many documents:

  → ratio is close to 1 → log is close to 0 → idf is very low

- Token appears only in a few documents:

  → ratio is close to 0 → log is large → idf is very high

Rare words carry more information. IDF captures this.

Putting It Together: TF-IDF

Multiply each TF value by the term's IDF:

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

$$\text{tfidf}(\text{“day”}, doc_4, D) = \frac{2}{3} \cdot \log\left(\frac{4}{1}\right) \approx 0.92$$

Take on me

$[0.1, 0.23, 0.23, 0,0,0,0,0]$

Take me on

$[0.1, 0.23, 0.23, 0,0,0,0,0]$

Take my breath away

$[0.07, 0,0, 0.35, 0.35,0.35,0,0]$

Day after day

$[0,0,0,0,0,0, 0.92, 0.46]$

What sklearn Does Under the Hood

sklearn's **`TfidfVectorizer`** uses the same idea but with different defaults:

Component	What we taught	sklearn default
TF	relative frequency: count / doc length	**raw count** (no division)
IDF	log(N / df)	log((1+N) / (1+df)) + 1 (smoothed)
Normalization	built into TF	**L2 normalization** of entire vector

sklearn vectorization in marimo notebook

04.

BM25

Where TF-IDF Falls Short (for retrieval)

1. **Linear term frequency**

- A word appearing 100 times gets 100 times the term frequency of a word appearing only once.
- But is mentioning "love" 100 times in a song really 100 times more informative than mentioning it once? At some point, more repetitions stop adding information.

2. **Document length normalization**

- Our relative term frequency (count / doc length) is an all-or-nothing approach. Sometimes you want partial normalization: a longer document naturally uses more words, but it also contains more information.
- Fully dividing by length might over-penalize longer documents.

BM25: TF-IDF with Two Knobs

BM25 (Best Matching 25, also called Okapi BM25) was developed in the 1990s at City University London for the Okapi information retrieval system.

- Is used for retrieval scoring and does not give a vector representation.
- It refines TF-IDF by adding two parameters:

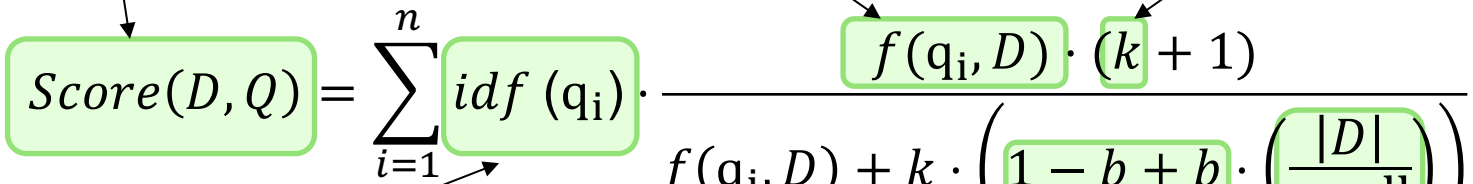
Parameter	Controls	Range
k	How quickly term frequency **saturates**	$k = 0$: ignore frequency $k \to \infty$: raw frequency Typical: 1.2 – 2.0
b	How much **document length** matters	$b = 0$: no length normalization $b = 1$: full normalization Typical: 0.75

The BM25 Formula

Final Score
Estimated relevance of document D to query Q

Term Frequency
Absolute number of times term q_i appears in document D

***k* Parameter**
Controls term frequency saturation



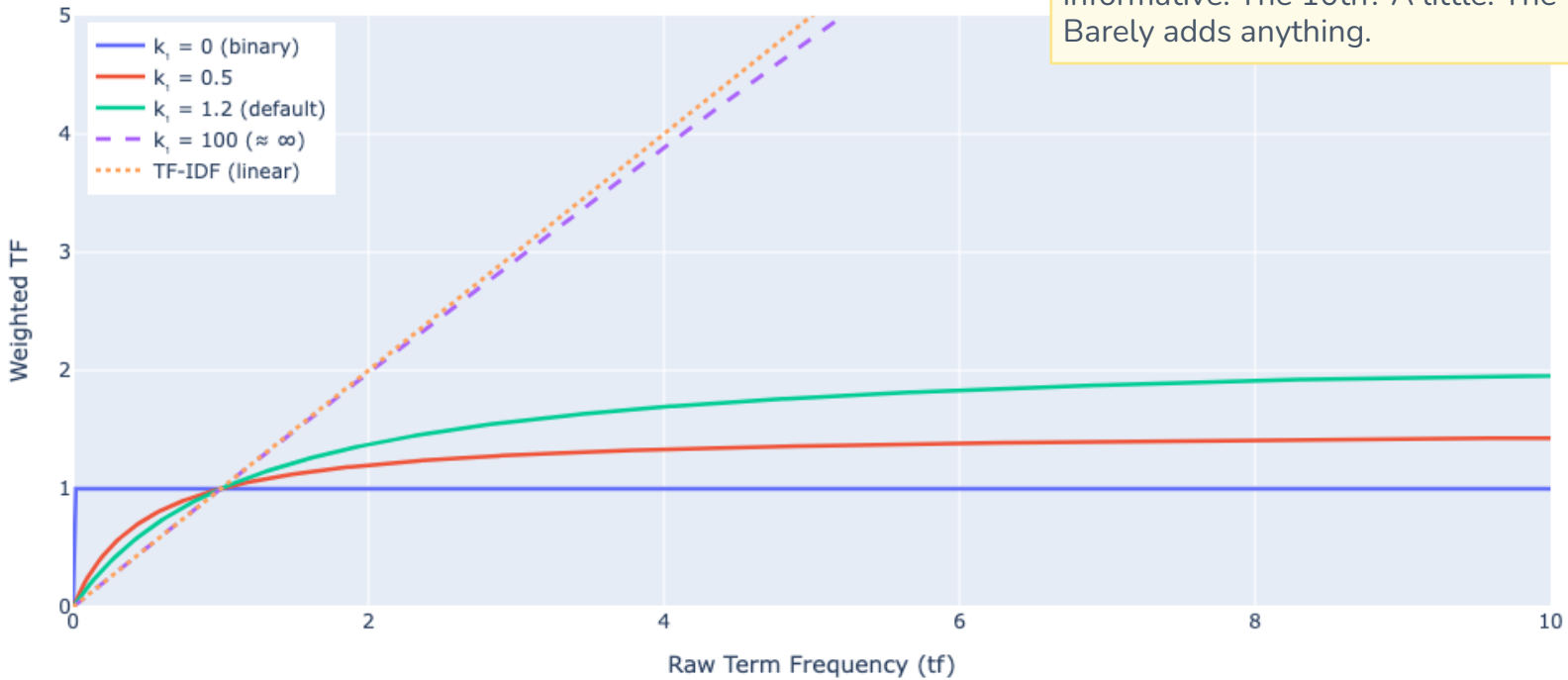
Inverse Document Frequency
Same idea as before: rare words get higher weight, common words get lower weight.

***b* Parameter**
Controls document length normalization to prevent bias towards longer documents

Average Document Length & Document Length
How long is the current document compared to the average document?

Saturation: The k Parameter

The Intuition
The 1st mention of a word is very informative. The 10th? A little. The 100th? Barely adds anything.



k = 0 → Binary
Word present/absent only. Score depends solely on IDF.

k = 1.2 → Recommended
Moderate saturation. 1st mention counts most; repeats give diminishing returns.

k → ∞ → Raw TF
Approaches linear TF-IDF. No saturation at all.

Length Normalization: The b Parameter

$$k \cdot \left(1 - b + b \cdot \left(\frac{|D|}{\text{avgdl}}\right)\right)$$

This adjusts the denominator based on how long the document is compared to the average document length avgdl.

- $b = 0$: The denominator simplifies to k. No length normalization at all.
- $b = 1$: Full normalization. A document twice as long as average gets penalized.
- $b = 0.75$ (typical): Partial normalization. Longer documents are penalized somewhat, but not fully.

Why partial? A longer song lyric has more words, but it also covers more ground. Fully penalizing length would be unfair.

Example

$$Score(D, \text{"take"}) = \text{idf}(\text{“take”}) \cdot \text{weighted\_tf}(\text{“take”}), k = 1.2, b = 0.75$$

Line	idf	weighted TF	BM25 score
"Take on me"	0.29	1.032	**0.30**
"Take me on"	0.29	1.032	**0.30**
"Take my breath away"	0.29	0.914	**0.26**
"Day after day"	—	0 (word absent)	**0**